

VETRA — Backend Implementation Guide

This is a separate document from `DOCUMENTATION.md` (which explains what the site does today). This one is a build plan: what to stand up so every feature currently simulated in the browser — auth, orders, chat, admin moderation, password resets, email verification — actually works against a real server. It's organized so you can build it in phases rather than all at once.

A real backend now exists in `backend/` at the project root, implementing everything through §7's step 5 below (auth, admin console, products/orders/cart, reports, reviews, the AI assistant). See `backend/README.md` for how to run it and what's still a marked `TODD` inside the code. The rest of this document is still the reference for *why* it's built the way it is and what comes next — read it alongside the code, not instead of it.

1. Where things stand today

The frontend is 100% static: HTML/CSS/JS, no build step. Before `backend/` existed, three areas faked persistence differently — this is now historical context for *why* the schema and API look the way they do, not the current state:

- **Public site + customer + vendor apps:** no persistence at all. Reload the page and everything resets. Mock data is hard-coded into the HTML/JS. (The frontend hasn't been rewired to call the new API yet — see the note at the end of §5.)
- **Admin console** (`admin/`): the one exception. It persisted to the browser's `localStorage` via a single data-access module, `admin/assets/data.js` (the global `VetraAdmin` object). Every admin page calls named functions on it (`getCustomers` , `setVendorStatus` , `resetCustomerPassword` , `inviteTeamMember` , etc.) instead of touching storage directly.

That second point is why this guide leaned on the admin console as the reference: `admin/assets/data.js` **was already shaped like an API client**. Its function names are exactly what `backend/src/routes/admin.routes.js` now implements for real — see §5's mapping table.

2. Target architecture

Chosen deployment target: Namecheap shared hosting (cPanel), for everything — frontend, backend, and database. This was picked because it's hosting already owned, at \$0 marginal cost, rather than adding a separate Render bill. It's a fine target for an MVP, but it's a *specific* environment with real constraints, and those shape several decisions below:

```
Browser (existing HTML/CSS/JS — unchanged, static files served straight off the hosting account)
|   HTTPS (JSON over REST)
▼
Node.js app, run via cPanel's "Setup Node.js App" (Passenger) — same account, same domain
|
|— MySQL/MariaDB (included free with cPanel) — users, orders, products, reports, activity log
```

- └ Local disk storage (cPanel file storage) – product images, avatars, chat attachments; move to an external object store (Cloudflare R2's free tier is a reasonable upgrade path) once storage or bandwidth starts to matter
- └ Email provider (Postmark / SendGrid / SES) – verification codes, password resets, order receipts
- └ SMS provider (Termii / Africa's Talking) – optional, for phone-based flows common in Nigeria
- └ Payment gateway (Pystack or Flutterwave) – real checkout, webhook-based (works fine here)

What this environment **doesn't** give you, and what that means: - **No WebSockets**. Passenger (cPanel's Node runner) doesn't reliably hold persistent connections, so real-time chat (`customer/chat.html`) has to wait — poll for new messages on an interval instead of pushing them, or defer chat's backend entirely until this moves off shared hosting. - **No Postgres, no pgvector** . MySQL/MariaDB only. This matters for the AI shopping assistant's product search (see §3's `reviews /embeddings` note) — instead of a vector-database similarity search, compute cosine similarity over stored embeddings in application code. That's genuinely fine at this catalog size (thousands, even tens of thousands of products is cheap to loop over in Node) and needs zero extra infrastructure. - **Shared CPU/RAM**, no root access, limited background job scheduling (cron jobs are supported via cPanel but constrained) — fine for request/response work, not for heavy background processing.

Session/auth: use JWTs sent in an `Authorization` header rather than server-side session cookies. This isn't just a style choice here — it sidesteps needing a `sessions` table and matches how the frontend and backend, even though they're on the same account/domain now, might not stay that way if parts of this move to another host later. **The token must carry the account's role**, because every route below needs to check it server-side — the current prototype only checks role in the browser, which is not a security boundary (see §6).

3. Data model

These map directly to what the front-end already renders, so field names below mostly match what `admin/assets/data.js` and the customer/vendor pages already expect. Written with MySQL/MariaDB in mind (the shared-hosting target from §2) — the two adjustments from a Postgres version of this same model: no native array type, so anything described as "array of X" below (`images` , `attachment_urls`) is a `JSON` column instead; and no native `UUID` type, so `id` is a `CHAR(36)` storing a UUID string (or `BINARY(16)` if you want it indexed more compactly — either works, `CHAR(36)` is simpler to reason about while building).

users (one table, a `role` column distinguishes buyer / vendor / admin)

Field	Notes
<code>id</code>	UUID, stored as <code>CHAR(36)</code>
<code>role</code>	<code>buyer</code> <code>vendor</code> <code>admin</code>
<code>name</code> , <code>email</code> , <code>phone</code> , <code>address</code>	<code>email</code> unique per role (a buyer and a vendor could share an email today in the mock data — decide if that's still allowed)
<code>password_hash</code>	<code>bcrypt/argon2</code> — never store plaintext, and note that today's "temporary password" reset flow needs a real hash written here
<code>status</code>	<code>active</code> <code>suspended</code> <code>pending</code> (vendors only, pre-approval) <code>rejected</code>
<code>signup_method</code>	<code>email</code> <code>google</code> etc.

Field	Notes
<code>last_login_at</code> , <code>created_at</code>	
<code>avatar_url</code>	pointer into object storage, not a base64 blob (the admin console's current avatar upload stores base64 in <code>localStorage</code> — fine for a demo, not for a database row)

Vendor-only fields (either a second `vendor_profiles` table keyed on `user_id` , or nullable columns on `users` if you'd rather keep one table): `store_name` , `category` , `description` , `products_count` , `orders_count` , `revenue_total` .

- `rating` / `review_count` : don't store these as columns to hand-maintain — compute them from `reviews` the same way the "Reviews" entry below already specifies (`AVG(rating)` , `COUNT(*)` grouped by `vendor_id`), or cache them on this row and recompute on every review write if the join becomes a measurable cost. This is what backs `store.html` 's "★ 4.8" stat, `vendors.html` 's directory cards, and the vendor mini-cards on `explore.html` 's Top Vendors rail — all three currently read a hand-typed `rating` field from the mock `assets/vendors.js` .
- `response_time` : the mock's "~10 min" stat has no real signal behind it yet (there's no messaging backend — see the `messages / conversations` note below). Either compute it once chat exists (median time between a buyer's first message and the vendor's first reply, over a rolling window), or drop the stat from the real UI rather than inventing a number — don't fabricate a metric with nothing behind it.
- `avatar_url` : same object-storage pointer convention as the buyer-facing `users.avatar_url` above, reused for the vendor's storefront avatar shown on `store.html` , `vendors.html` , and the Top Vendors rail.

Payout account (backs `vendor/earnings.html` 's Payout Account section, `vendor/assets/payout.js`): `payout_bank_name` , `payout_account_number` , `payout_account_name` . **Never store the raw account number in plaintext if you can avoid it** — encrypt it at rest (or store only a tokenized reference from whatever payout processor you integrate, e.g. Paystack's transfer-recipient API) and never return the full number in any API response once it's saved; the demo's masked-display convention (`•••• 6789`) is the UI contract a real backend needs to actually enforce server-side, not just hide client-side.

Admin-only fields: `admin_role` (`Super Admin` | `Moderator` | `Support`) — keep this distinct from the top-level `role` column (which is just "this is an admin account"); `admin_role` is what the current permission checks (`isSuperAdmin()` , `getVisibleActivity()`) key off.

products

`id` , `vendor_id` , `name` , `category` , `price` , `stock_quantity` , `description` , `images` (JSON array of object-storage/local-disk URLs), `video_url` (nullable), `status` (`active` / `out_of_stock` / `removed`), timestamps. This backs the vendor "Add Product" modal, `vendor/products.html` , and the customer-facing product grids.

- `description_embedding` : JSON column storing the embedding vector (an array of ~1,500 floats) generated once from the product's name+description, used for the AI shopping assistant's search — see §2's note on computing similarity in application code instead of `pgvector` . Regenerate it whenever `name` or `description` changes; leave it `NULL` until then so the search step can just skip un-embedded rows.

Product badges ("New" / "Hot")

Not a separate table — two columns on `products` plus logic the front-end already has, ready to point at real data. `customer/assets/products.js` 's `getProductBadge(product)` decides the badge from exactly two fields: `product.createdAt` and `product.salesCount`. Right now those are hand-typed demo values (see that file's `PRODUCTS` object); a real backend needs to make them real without touching that function at all:

- **created_at** — `products` already needs this column for ordinary row bookkeeping; nothing extra to add. `getProductBadge()` 's "New" check is `ageDays <= 21` — tune `BADGE_NEW_WINDOW_DAYS` in `products.js` if 21 days isn't right, but the check itself doesn't change.
- **sales_count** — `COUNT(*)` (or `SUM(quantity)` , if "sold 50 units" should count more than "sold in 50 separate orders") from `order_items` joined to `orders` where `orders.status` is a completed/non-cancelled state, grouped by `product_id` . Two implementation options, in order of preference: (1) compute it live with that aggregate query whenever `GET /api/products` / `GET /api/products/:id` runs — simplest, and correct by construction, at the cost of a join on every product list request; (2) maintain it as a denormalized counter column on `products` , incremented when an order's status flips to its "counts as sold" state (in the same place `escrow_status` gets updated) — faster to read, but now something that can drift if an increment is ever missed, so only worth it once product-list latency actually matters. Start with (1).
- **Wiring it up:** have `GET /api/products` and `GET /api/products/:id` include `created_at` / `sales_count` (or `createdAt` / `salesCount` if the API camel-cases its JSON — match whatever the rest of the API does) in the response, and nothing else needs to change — `products.js` 's `PRODUCTS` object stops being hard-coded and becomes "whatever the last `GET /api/products` call returned," `getProductBadge()` keeps running exactly as it does today, and `assets/badges.js` 's `decorateProductBadges()` keeps applying its result to the DOM unchanged. Resist the temptation to compute the badge server-side and return a `badge: "new"` field instead — that duplicates the threshold logic in two languages/places, and the one place is doing fine.

orders and order_items

An order belongs to a buyer, has a delivery/pickup choice, a status, and a total; `order_items` line-items reference `products` with quantity and price-at-purchase. This is what `customer/cart.html` 's checkout, `customer/orders.html` 's tracking view, and `vendor/orders.html` 's shipment-update modal all need — right now `cart/order` contents live only in the DOM.

- `status` : `pending` → `processing` → `shipped` → `out_for_delivery` → `completed` (or `cancelled` from any state before delivery). The customer/vendor UIs already show all six states; the extra granularity (`shipped` , `out_for_delivery`) beyond a simpler `pending/processing/completed/cancelled` set is what backs the delivery-tracking timeline on `customer/orders.html` .
- `carrier` , `tracking_number` : set by the vendor (currently via `vendor/orders.html` 's Update Shipment modal, in-memory only). Nullable until shipped.
- `status_history` : either a separate `order_status_events` table (`order_id` , `status` , `changed_at` , `changed_by_user_id`) or timestamp columns per status (`shipped_at` , `out_for_delivery_at` , `delivered_at`) — either backs the step-by-step timeline `customer/orders.html` currently renders from hard-coded per-step timestamps.
- `escrow_status` / `escrow_released_at` : matches the hold-until-confirmed mechanics described on `buyer-protection.html` (funds release on buyer confirmation or 48hrs after delivery, whichever comes first) and `vendor-protection.html` .

reports

`id` , `type` (`customer` / `vendor` / `product`), `target_id` , `reporter` (free text or a `reporter_user_id` if reports should always come from a logged-in account), `reason` , `status` (`open` / `resolved` / `dismissed`),

`attended_by_user_id` , `attended_at` , `created_at` . Matches `admin/assets/data.js` 's `reports` shape exactly.

activity_log

Append-only: `id` , `type` (`account` / `vendor` / `report` / `order` / `login`), `message` (or better — structured fields you render into a message client-side, rather than pre-baked HTML strings like the mock does), `actor_user_id` (nullable — null for platform/system events), `target_type` , `target_id` , `created_at` . This one table is what powers the admin dashboard's recent-activity feed, the full activity log page, and every "who did this" attribution shown on reports and account detail pages. Insert a row from every mutating admin action server-side — don't rely on the client to log its own actions, or a compromised/buggy client can go unaudited.

admin_invites

`id` , `name` , `email` , `role` , `invited_by_user_id` , `verification_code_hash` (hash it, don't store the raw code once you're sending real email), `expires_at` , `status` (`pending` / `verified` / `cancelled`). Backs the invite-and-verify admin onboarding flow.

site_banners — planned, not yet in `backend/migrations/001_init.sql`

Proposed shape, added to this guide this session but not yet to the actual schema: `id` , `image_url` (object-storage/CDN URL — the mock's `admin/assets/data.js` version stores a base64 data URL instead, same "no real file storage yet" caveat as `users.avatar_url`), `alt_text` , `display_order` (int — carousel order; the mock reorders by mutating array position, a real table needs an explicit column to `ORDER BY`), `created_at` . Backs `admin/settings.html` 's **Site Banners** card (add/remove/reorder) and the picture-only promo carousel both `customer/dashboard.html` and `customer/explore.html` render from it — see §5's `/api/site-banners` routes (also 🕒 planned, matching this table). Deliberately not modeling anything beyond an ordered image list: there's no title/subtitle/link-target field, because the frontend feature this backs is intentionally picture-only (an earlier version had per-slide text, removed by request — don't bring it back here just because a real table could support it).

report_evidence

New table backing `vendor/orders.html` 's "Submit evidence" modal: `id` , `report_id` , `vendor_user_id` , `response_text` , `attachment_urls` (array, object storage), `submitted_at` . A report can have zero or more of these; admin's `reports.html` should surface them when reviewing a report so a vendor's response is actually read before a decision is made — right now the mock version just flips the card to an "awaiting review" state with nothing behind it.

reviews

`id` , `vendor_id` , `buyer_id` , `order_id` (**required, not nullable** — this is what makes "verified purchase" real instead of a UI label), `rating` (1–5), `review_text` , `created_at` — see `backend/migrations/001_init.sql` , which also adds a `UNIQUE KEY uniq_review_per_order (order_id)` and a `CHECK (rating BETWEEN 1 AND 5)` constraint, enforcing "one review per order" and the rating range at the database level, not just in application code. `customer/store.html` 's review form currently has no way to check "did this buyer complete an order with this vendor" since there's no backend — the real version should reject a review submission server-side unless a `completed` order exists linking that `buyer_id` and `vendor_id` , and should look up `order_id` automatically rather than trusting anything the client sends. Aggregate rating (shown as

the "4.7 · 3 reviews" summary) should be computed server-side (or cached on `vendor_profiles` and recomputed on write), not summed client-side over every review on every page load.

messages / conversations (chat)

Not fully speced here since the front-end for these (`customer/chat.html`) is UI-only mock data — but a `conversations` table (buyer_id, vendor_id) plus a `messages` table (conversation_id, sender_id, body, attachment_url, created_at) is the standard shape. Real-time delivery needs the WebSocket layer from §2.

4. Authentication & the three account types

1. **Signup/signin** (`signup.html` , `signin.html`): real password hashing, email verification before first login (or at least before checkout, to cut down on fraud), and a real session/JWT issued on success. Buyer and vendor signup should almost certainly be separate flows hitting the same `users` table with `role` set accordingly.
 2. **Admin signin** (`admin/login.html`): currently just an email lookup with no password check at all — this is the single most important gap to close. Real build: admin accounts still live in `users` (role= `admin`), get a real password, and — given how sensitive this surface is — should support 2FA (the console already has a "Two-factor authentication" toggle in Settings that's currently cosmetic).
 3. **Session propagation**: once real sessions exist, the front-end's `VetraAdmin.getCurrentAdmin()` simulation (which just reads whichever admin last "signed in" via email lookup) gets replaced by "whoever the session belongs to" — the server already knows this from the auth cookie/token on every request, so the client no longer needs to manage it at all.
 4. **Guest checkout**: `admin/settings.html` 's "Allow guest checkout" toggle needs a real effect — when it's on, the checkout endpoint should accept an order without a `buyer_id` (or create a lightweight guest record keyed by email/phone for order tracking), and when it's off, checkout should require a session.
 5. **Password change (`customer/settings.html` 's Security card) needs its own auth check the current UI doesn't collect**. This session's UI change removed the "Current password" field from that form (a UX call — the field verified nothing against a backend that doesn't exist yet), leaving only "New password"/"Confirm new password". **Don't mirror that omission server-side**. A real `PATCH /api/auth/password` (or similar) endpoint must still confirm the request is really from the account holder before accepting a new password — either by requiring the current password in the body after all (add the field back for this one real endpoint, even though the demo UI doesn't have it) or by requiring a fresh re-authentication (a recent login timestamp on the JWT, or a short-lived step-up token) if the product decision is to keep the field gone from the UI. Changing a password with no verification at all is a real account-takeover hole the moment a session token leaks, so this is one of the few places where the shipped frontend and the correct backend behavior can't just be a 1:1 mapping — flag it back to whoever owns the UI if the field's removal wasn't meant to imply "skip verification," not just silently work around it here.
-

5. API surface, mapped to existing front-end calls

Everything marked  **built** below exists right now in `backend/src/routes/` and was smoke-tested (server boot, route mounting, auth rejection, and graceful DB-error handling — see `backend/README.md` 's "What's been checked" section; this is a manual sanity check, not automated test coverage).  **planned** means it's in the

schema/plan but not implemented yet, usually because it depends on a provider that isn't configured (email) or infrastructure not worth building before there's real usage (a scheduled job for escrow auto-release).

Every route below goes through two shared pieces first, so they're not repeated per endpoint: - **asyncHandler** (backend/src/utils/asyncHandler.js) wraps every handler so a thrown/rejected error reaches **errorHandler** instead of crashing the process. - **errorHandler** (backend/src/middleware/errorHandler.js) is the last middleware in the chain: a MySQL duplicate-key error (ER_DUP_ENTRY) becomes 409 { "error": "That already exists." } ; anything else becomes the handler's own res.status(...).json({error: ...}) if it set one, or a generic 500 { "error": "Something went wrong." } with the real error only logged server-side, never sent to the client.

Auth on a route is one of: **public** (no token needed), **optionalAuth** (Authorization: Bearer <token> read if present, request proceeds either way — req.user is undefined for a guest), or a **required** role — **requireAuth** first (401 { "error": "Missing bearer token." } or { "error": "Invalid or expired token." } if absent/bad), then **requireRole**("buyer"|"vendor"|"admin") (403 { "error": "Not allowed for this account type." }) or, for three admin-only routes (DELETE /api/admin/team/:id , POST /api/admin/invites , POST /api/admin/invites/:id/verify), **requireAdminRole**("Super Admin") (403 { "error": "Not allowed for this admin role." }).

/api/auth (public — backend/src/routes/auth.routes.js)

POST /api/auth/signup - Body: { role: "buyer"|"vendor", name, email, password, phone?, storeName?, storeCategory? } — storeName is required when role is "vendor" . - 201 : { token, user: { id, role, name, email, status } } . status is "pending" for a new vendor (matches admin/vendors.html 's Pending Approval queue), "active" for a buyer. - 400 : role missing/invalid; name / email / password missing; vendor signup missing storeName . - Side effect: a vendor signup also writes an activity_log row (type: "vendor" , no actor_user_id — nobody on staff did this) so it shows up in admin/activity.html 's feed immediately. - Password is hashed with bcryptjs (10 salt rounds, backend/src/utils/password.js) before the insert — never stored or logged in plaintext.

POST /api/auth/signin - Body: { role: "buyer"|"vendor", email, password } . - 200 : { token, user: { id, role, name, email, status } } . - 400 : role missing/invalid. - 401 : { "error": "Incorrect email or password." } — no email/password mismatch is distinguished in the response, to avoid leaking which part was wrong. - 403 : { "error": "This account has been suspended. Contact support." } if status = "suspended" . - Side effects: updates last_login_at ; writes a login -type activity row.

POST /api/auth/admin-signin - Body: { email, password } — no role field, since this route only ever looks at role = 'admin' rows. - 200 : { token, user: { id, name, email, adminRole } } . - 401 : same generic incorrect-credentials message as buyer/vendor signin. - Side effect: activity row with actor_user_id set to the admin's own id (so "who signed in" is attributable, matching the existing admin console's login-feed entries).

The JWT itself (backend/src/utils/jwt.js) is signed with { id, role, adminRole } as the payload (adminRole is null for buyer/vendor tokens) and expires per JWT_EXPIRES_IN in .env (default 7d). Every subsequent authenticated request reads this payload back as req.user via requireAuth / optionalAuth .

PATCH /api/auth/password — **planned, not yet built.** requireAuth , any role. - Body: { currentPassword, newPassword } — see §4 point 5 for why currentPassword is required here even though customer/settings.html 's form no longer collects it; the endpoint needs it (or an equivalent re-auth check) regardless of what the form sends today. - 200 : { ok: true } . - 400 : { "error": "currentPassword and newPassword are required." } , or a weak/too-short newPassword per whatever minimum length policy is

chosen. - 401: {"error": "Current password is incorrect."} — bcrypt.compare(currentPassword, user.password_hash) fails. - Side effect: writes an account -type activity row (self-attributed) so a password change is auditable like every other account action; consider invalidating other active sessions/tokens for the account, since a leaked token is exactly the scenario this endpoint exists to recover from.

/api/products (backend/src/routes/products.routes.js)

GET /api/products — public. - Query params (all optional, combine with AND): ?vendor=<id> , ?category=<name> , ?q=<text> (matches name or description via LIKE %text%). - 200: array of { id, vendor_id, name, category, price, stock_quantity, images, status, created_at } for status = 'active' rows only, newest first (backend/src/routes/products.routes.js). **sales_count is not in this response yet** — §3's "Product badges" section already specs the aggregate query and column this route should add it from, and created_at (already returned) is half of what getProductBadge() needs, but the sales_count half hasn't been wired up on either the schema or this route. Add it here once it exists.

GET /api/products/:id — public. - 200: the full product row — currently every column MySQL's products table actually has (a plain SELECT *), which does **not** include sales_count for the same reason as above. - 404: {"error": "Product not found."}.

POST /api/products — requires requireAuth + requireRole("vendor") .- Body: { name, category, price, stockQuantity?, description?, images?, videoUrl? } — images is a plain array of URL strings, stored as JSON .- 201: { id } .- 400: {"error": "name, category, and price are required."} .- vendor_id is taken from the authenticated token (req.user.id), never from the request body — a vendor cannot create a product on another vendor's behalf by passing a different id.

PATCH /api/products/:id — vendor-only, ownership-checked. - Body: any subset of { name, category, price, stockQuantity, description, status, images } — only the fields present are updated. - 200: { ok: true } .- 404: product doesn't exist. 403: {"error": "You don't own this product."} if vendor_id on the row doesn't match the token. 400: {"error": "No fields to update."} if the body was empty.

DELETE /api/products/:id — vendor-only, ownership-checked. - 200: { ok: true } . Same 404/403 as PATCH. - This is a **soft delete** — it sets status = 'removed' rather than deleting the row, specifically so existing order_items rows (which foreign-key to products.id) don't break for past orders.

/api/vendors — planned, not yet built

Not in backend/ yet, and not covered by any existing route: admin/vendors.html 's backing endpoint (GET /api/admin/vendors) is admin-only and requires a token, but customer/store.html (a public storefront page) and customer/vendors.html (the vendor directory added this session, with its name-search box) both need a **public** way to list/look up vendors. Right now both pages read the hard-coded assets/vendors.js mock instead — this is the endpoint that replaces it.

GET /api/vendors — public. - Query params: ?q=<text> (matches store_name via LIKE %text%, case-insensitive — this is what vendors.html 's #vendorSearchInput should call as the user types, instead of filtering an in-memory array client-side once real data exists). - 200: array of { id, store_name, avatar_url, category, status, rating, review_count, address } for status = 'active' vendor rows only (a pending or rejected vendor shouldn't be publicly browsable). This is the exact shape vendors.html 's directory cards and explore.html 's Top Vendors rail both need.

GET /api/vendors/:id — public. - **200** : the full vendor profile — everything above plus `description / bio` , `products_count` , `orders_count` , `member_since` (`users.created_at`) — this is what `store.html` renders. - **404** : `{ "error": "Vendor not found." }` , or if the vendor's `status` isn't `active` (don't distinguish "doesn't exist" from "exists but suspended" in the response — same reasoning as the auth error messages in §4 not leaking which part of a login failed).

/api/site-banners — planned, not yet built

Backs `admin/settings.html` 's **Site Banners** card and the picture-only promo carousel on `customer/dashboard.html / customer/explore.html` (both added this session). Split public-read/admin-write, same shape as `/api/vendors` above.

GET /api/site-banners — public, no auth. - **200** : array of `{ id, imageUrl, alt }` , ordered by `display_order` ascending. This is exactly what `dashboard.html / explore.html` 's inline script should `fetch()` on load instead of calling `VetraAdmin.getSiteBanners()` against `admin/assets/data.js` 's `localStorage` directly — closing the one thing about this feature that doesn't fit the rest of the API-based architecture (see the callout at the end of §5: reaching into admin's `localStorage` from a customer page is a stopgap, not the intended long-term shape, and this route is what removes the need for it here same as everywhere else). - Empty array (not an error) if no banners are set — the frontend already handles this (falls back to its own `DEFAULT_BANNERS` constant rather than rendering nothing).

POST /api/site-banners — `requireAuth + requireRole("admin")` .- Body: `{ imageUrl, alt? }` . In practice `imageUrl` comes from a prior file-upload step (§7 step 8), not a raw URL typed into a form — the current mock's `FileReader` -to-base64 stands in for that upload step exactly like the admin avatar photo does. - **201** : `{ id }` . New banners append to the end (`display_order = MAX(display_order) + 1`). - **400** : `{ "error": "imageUrl is required." }` .

PATCH /api/site-banners/:id/order — `requireAuth + requireRole("admin")` .- Body: `{ direction: "up"|"down" }` — swaps `display_order` with the adjacent row, mirroring `VetraAdmin.moveSiteBanner()` 's array-swap exactly rather than accepting an arbitrary new position (simpler, and a real drag-to-reorder UI can still be built on top of repeated up/down calls, or this route can grow a `{ position: N }` variant later if that's ever needed). - **200** : `{ ok: true }` . **400** if already at that end of the list (nothing to swap with) — matches the mock's disabled-button-at-the-edge behavior.

DELETE /api/site-banners/:id — `requireAuth + requireRole("admin")` .- **200** : `{ ok: true }` . **404** if not found. Hard delete is fine here (unlike `products` ' soft delete) — nothing else foreign-keys to a banner row. - Side effect on every write above: an `account` -type `activity_log` row, same convention as `VetraAdmin.addSiteBanner()` / `removeSiteBanner()` already follow in the mock.

/api/orders (backend/src/routes/orders.routes.js)

POST /api/orders — checkout. `optionalAuth` (works signed-in or as a guest). - Body: `{ vendorId, items: [{ productId, quantity? }], deliveryMethod?: "delivery"|"pickup", deliveryAddress?, guest?: { name, email, phone } }` — `guest` is required (all three sub-fields) only when there's no bearer token; `quantity` defaults to 1 per item. - **201** : `{ id, total, status: "pending" }` . `total` is computed server-side from the current `products.price` for each item — never trusted from the client. - **400** : missing `vendorId` /empty `items` ; guest checkout missing name/email/phone; a `productId` that doesn't exist or isn't `status = 'active'` ; insufficient `stock_quantity` for any line item. - All the writes (the `orders` row, every `order_items` row, and each product's `stock_quantity` decrement) happen inside **one database transaction** — if any step fails, everything rolls back rather than leaving a half-created order with mismatched stock. - Side

effect: an `order` -type activity row, with `actor_user_id` set only when signed in (`null` for a guest checkout, same reasoning as the report-filing pattern below — nobody on staff acted, and a guest isn't an admin either).

GET `/api/orders/mine` — customer order history. `requireAuth` + `requireRole("buyer")` . - `200` : every order for the signed-in buyer, newest first, each row joined with `vendor_name` (the vendor's `store_name`) — this is what `customer/orders.html` 's list and its per-order tracking timeline render from.

GET `/api/orders/vendor` — `requireAuth` + `requireRole("vendor")` . - Query: optional `?status=<one of the six statuses>|all` . - `200` : every order for the signed-in vendor (optionally filtered), newest first.

PATCH `/api/orders/:id/shipment` — `requireAuth` + `requireRole("vendor")` , ownership-checked. - Body: `{ status: "pending"|"processing"|"shipped"|"out_for_delivery"|"completed"|"cancelled", carrier?, trackingNumber? }` . - `200` : `{ ok: true }` . - `400` : `{"error": "status must be one of: pending, processing, shipped, out_for_delivery, completed, cancelled"}` . `404` : order doesn't exist. `403` : `{"error": "This isn't your order."}` if the order's `vendor_id` doesn't match the token. - Side effects: stamps the matching timestamp column (`shipped_at` / `out_for_delivery_at` / `delivered_at` / `cancelled_at` — `pending` / `processing` stamp nothing extra); setting `status: "completed"` additionally sets `escrow_status = 'released'` and `escrow_released_at = NOW()` , modeling the "funds release on delivery" mechanic from `buyer-protection.html` . The 48-hour no-response auto-release case isn't handled here — it needs a scheduled job, not something a single request can do (see §7). - Writes an `order` -type activity row attributed to the vendor.

`/api/vendors/:vendorId/reviews` (`backend/src/routes/reviews.routes.js` , mounted with `mergeParams`)

GET `/api/vendors/:vendorId/reviews` — public. - `200` : `{ average: number|null, count: number, reviews: [{ id, rating, review_text, created_at, buyer_name }] }` , newest first. `average` is `null` (not `0`) when there are zero reviews, so the frontend can distinguish "no reviews yet" from "reviews exist and are bad."

POST `/api/vendors/:vendorId/reviews` — `requireAuth` + `requireRole("buyer")` . - Body: `{ rating: 1-5, text, orderId }` . - `201` : `{ id }` . - `400` : `{"error": "rating (1-5) and text are required."}` . - `403` : `{"error": "You can only review a vendor after a completed order."}` — this is the real "verified purchase" check: it looks up an order matching `id = orderId AND buyer_id = <token> AND vendor_id = :vendorId AND status = 'completed'` , and refuses if none exists. The client cannot fake this by passing an arbitrary `orderId` . - `409` (via the shared `ER_DUP_ENTRY` handler): a second review attempt on the same `orderId` — the schema's `UNIQUE KEY uniq_review_per_order` enforces one review per order at the database level, not just in application logic.

`/api/reports` (`backend/src/routes/reports.routes.js`) and evidence

Every route here requires `requireAuth` first; role checks follow per-route.

GET `/api/reports` — `requireRole("admin")` . - Query: optional `?status=open|resolved|dismissed|all` . - `200` : the full report list (every field, every type), newest first — this is admin's entire moderation queue, `admin/reports.html` .

GET `/api/reports/mine` — `requireRole("vendor")` . - `200` : only reports where `type = 'vendor'` AND `target_id = <token's id>` , each with an `evidence_ids` field (a comma-joined list of `report_evidence.id` values from a `LEFT JOIN + GROUP_CONCAT` , `null` if none submitted yet) — this backs `vendor/orders.html` 's read-only "Reports against your store" panel.

PATCH /api/reports/:id/status — `requireRole("admin")` . - Body: `{ status: "resolved"|"dismissed" }` . - `200` : `{ ok: true }` . `400` : invalid status value. - Side effects: sets `attended_by_user_id` to the **authenticated admin's own id** (never accepted from the request body — a moderator can't credit the resolution to someone else) and `attended_at = NOW()` ; writes a `report` -type activity row.

POST /api/reports/:id/evidence — `requireRole("vendor")` . - Body: `{ responseText, attachmentUrls? }` (`attachmentUrls` is an array of URL strings). - `201` : `{ id }` . `400` : missing `responseText` . `404` : the report doesn't exist, isn't type `"vendor"` , or isn't against *this* vendor (all three collapse into one 404 rather than distinguishing them, so a vendor can't probe for the existence of another vendor's report by id). - This is a pure **append** — it does not change the parent report's `status` ; only an admin's `PATCH .../status` call does that.

Not yet built (flagged in §5's summary table too): `POST /api/reports` for a buyer to originate a new report from an order. The current frontend demo (`customer/assets/report-issue.js`) calls `admin/assets/data.js` 's `VetraAdmin.addReport()` directly instead, since this endpoint doesn't exist yet — see the table below for the intended shape.

/api/admin (backend/src/routes/admin.routes.js)

Every route requires `requireAuth` + `requireRole("admin")` (applied once via `router.use()` at the top of the file) — individual routes layer `requireAdminRole("Super Admin")` on top where noted.

GET /api/admin/customers — optional `?q=<text>` (matches name or email). `200` : array of `{ id, name, email, phone, address, status, signup_method, last_login_at, created_at }` .

GET /api/admin/customers/:id — `200` : the same shape, one row. `404` if not found or not a buyer.

PATCH /api/admin/customers/:id/status — body `{ status: "active"|"suspended", reason? }` . `200` : `{ ok: true }` . `400` invalid status, `404` not found. Writes an `account` -type activity row ("`Suspended`"/"`Reactivated`" + the reason if given).

POST /api/admin/customers/:id/reset-password — no body needed. `200` : `{ ok: true, message: "Reset link sent to the account holder." }` . Generates a random 24-byte hex token — **currently logged to the server console, not emailed** (`console.log("[password-reset] ...")`), since there's no email provider wired up yet; this is the one deliberately incomplete piece flagged in `backend/README.md` . Writes an activity row either way, so the *attempt* is auditable even before email delivery exists.

GET /api/admin/vendors — optional `?status=active|pending|suspended|rejected|all` . `200` : array of `{ id, name, email, phone, address, store_name, store_category, status, last_login_at, created_at }` .

GET /api/admin/vendors/:id — same shape plus `store_description` , one row. `404` if not found.

PATCH /api/admin/vendors/:id/status — body `{ status: "active"|"suspended"|"rejected", reason? }` . `200` : `{ ok: true }` . Activity message verb is picked from the status (`Approved` / `Suspended` / `Rejected`) — note there's no explicit "`pending→active`" vs. "`suspended→active`" distinction server-side, both just say "`Approved`" today since the verb table only keys off the *new* status, not the transition; a nitpick worth fixing if the activity feed's wording matters (the old prototype's `admin/assets/data.js` version explicitly checked `prevStatus === "pending"` to say "`Approved`" vs. "`Reactivated`" — this route doesn't yet).

POST /api/admin/vendors/:id/reset-password — identical shape/behavior to the customer version above.

GET /api/admin/stats — **200**: { totalCustomers, totalVendors, suspendedAccounts, openReports, platformOrders, platformRevenue }, every number computed with a real `COUNT / SUM` query at request time (`platformRevenue` sums `orders.total` where `status = 'completed'`, `COALESCE` 'd to `0` so an empty table returns `0` rather than `null`).

GET /api/admin/activity — optional `?adminId=<id>` (**Super Admin only** — silently ignored for other roles, since their query is already scoped). **200**: up to 200 rows, newest first, each joined with the actor's `name` as `actor_name` (`null` for system events). **Role-scoped server-side**: a Super Admin gets every row (or just one admin's, with `?adminId=`); a Moderator/Support admin's query is forced to `actor_user_id IS NULL OR actor_user_id = <their own id>` regardless of what they pass — they cannot see another admin's actions by querying directly, unlike the original prototype's version of this rule which only filtered client-side.

GET /api/admin/team — **200**: array of { `id`, `name`, `email`, `admin_role`, `avatar_url` }, oldest-first (so the original Super Admin tends to sort first).

DELETE /api/admin/team/:id — **requireAdminRole("Super Admin")**. **200**: { `ok`: `true` }. **404** if not an admin. **400**: {`"error"`: `"Can't remove the platform's last Super Admin."`} — checked by counting `admin_role = 'Super Admin'` rows before allowing the delete, so the console can never end up with zero full-access admins.

POST /api/admin/invites — **requireAdminRole("Super Admin")**. Body: { `name`, `email`, `adminRole`: `"Super Admin"|"Moderator"|"Support"` }. **201**: { `id` }. **400**: any field missing/invalid. Generates a random 6-digit code (`crypto.randomInt(100000, 999999)`), stores only its `bcrypt` hash plus a 15-minute expiry — **the raw code is currently logged to the server console** (`[admin-invite] ...`), not emailed, same TODO pattern as password resets.

POST /api/admin/invites/:id/verify — **requireAdminRole("Super Admin")**. Body: { `code` }. **201**: { `userId`, `tempPassword` } — the temp password is returned in the response body only because there's no email step to send it through yet; a real deployment should email it and never put it in an API response. **404**: invite not found or already used. **400**: {`"error"`: `"This code has expired."`} (past `expires_at`) or {`"error"`: `"Incorrect code."`} (`bcrypt.compare` fails). On success: creates the new admin `users` row, marks the invite `verified`, and logs an `account`-type activity row.

/api/assistant (backend/src/routes/assistant.routes.js)

POST /api/assistant/chat — `optionalAuth` (works signed in or anonymously; nothing in the current logic actually branches on `req.user`, it's just there for when personalization is added later). - Body: { `message`, `history?: [{ role: "user"|"assistant", content }]` } — `history` is the prior turns of the conversation, passed straight through to Claude as-is so the frontend owns conversation state, not this endpoint. - **200**: { `reply`: `string`, `matchedProducts`: [{ `id`, `vendor_id`, `name`, `category`, `price`, `stock_quantity` }] }. - **400**: {`"error"`: `"message is required."`} - What happens server-side: `findCandidateProducts()` lowercases the message, strips everything except letters/digits/`#`/whitespace, splits on whitespace, drops words ≤ 2 chars and a small stopword list (`a`, `an`, `the`, `for`, `with`, `and`, `or`, `of`, `to`, `me`, `i`, `want`, `need`), then runs one `LIKE`-based SQL query OR-ing every remaining keyword against `name / description / category`, capped at 8 results. Those candidates (name, category, price formatted as `$12,345`, and `id`) are interpolated into a system prompt that explicitly instructs the model to **only** recommend from that list and never invent a product/price/vendor, then sent to `claude-haiku-4-5-20251001` (overridable via `ASSISTANT_MODEL` in `.env`) with `max_tokens`: **400**. The reply text is extracted from the response's `content` blocks (filtering to `type === "text"`, joining any

that exist) — this correctly handles the case where a model response has multiple text blocks, though in practice a simple chat completion like this almost always returns exactly one. - This intentionally has **no rate limiting or per-user cost caps** yet — worth adding before any real traffic, since every call is a paid Anthropic API request (see the earlier cost discussion in this project's history for rough per-conversation pricing).

Summary table (quick reference — see above for full request/response detail)

Admin — the direct swap-in for `admin/assets/data.js`

Mock function	Real endpoint	Status
<code>getCustomers()</code> / <code>getCustomer(id)</code>	<code>GET /api/admin/customers</code> , <code>GET /api/admin/customers/:id</code>	✅ built
<code>setCustomerStatus(id, status, reason)</code>	<code>PATCH /api/admin/customers/:id/status</code>	✅ built
<code>resetCustomerPassword(id)</code>	<code>POST /api/admin/customers/:id/reset-password</code>	✅ built, email TODO
<code>getVendors()</code> / <code>getVendor(id)</code>	<code>GET /api/admin/vendors</code> , <code>GET /api/admin/vendors/:id</code>	✅ built
<code>setVendorStatus(id, status, reason)</code>	<code>PATCH /api/admin/vendors/:id/status</code>	✅ built
<code>resetVendorPassword(id)</code>	<code>POST /api/admin/vendors/:id/reset-password</code>	✅ built, email TODO
<code>getReports()</code>	<code>GET /api/reports</code> (optional <code>?status=</code>)	✅ built
<code>setReportStatus(id, status)</code>	<code>PATCH /api/reports/:id/status</code>	✅ built
<code>getActivity()</code> / <code>getVisibleActivity()</code>	<code>GET /api/admin/activity</code> (Super Admin: optional <code>?adminId=</code>)	✅ built
<code>getTeam()</code>	<code>GET /api/admin/team</code>	✅ built
<code>setTeamMemberAvatar(id, dataUrl)</code>	<code>POST /api/admin/team/:id/avatar</code>	🕒 planned — needs file uploads (\$7 step 8)
<code>removeTeamMember(id)</code>	<code>DELETE /api/admin/team/:id</code>	✅ built
<code>inviteTeamMember()</code> / <code>verifyTeamInvite()</code>	<code>POST /api/admin/invites</code> , <code>POST /api/admin/invites/:id/verify</code>	✅ built, email TODO
<code>resendInviteCode()</code> / <code>cancelInvite()</code>	<code>POST /api/admin/invites/:id/resend</code> , <code>DELETE /api/admin/invites/:id</code>	🕒 planned
<code>getStats()</code>	<code>GET /api/admin/stats</code>	✅ built
<code>resetDemoData()</code>	dropped — prototype-only concept	N/A

Everything else

What it's for	Real endpoint	Status
Buyer/vendor signup	POST /api/auth/signup	✅ built
Buyer/vendor signin	POST /api/auth/signin	✅ built
Admin signin	POST /api/auth/admin-signin	✅ built
Buyer/vendor password change (settings.html 's Security card)	PATCH /api/auth/password	🕒 planned
Browse/search products	GET /api/products , GET /api/products/:id	✅ built
Browse/search vendors (vendors.html , store.html)	GET /api/vendors (optional ?q=), GET /api/vendors/:id	🕒 planned
Site banner carousel (dashboard.html , explore.html)	GET /api/site-banners ; admin: POST /api/site-banners , PATCH /api/site-banners/:id/order , DELETE /api/site-banners/:id	🕒 planned
Vendor product CRUD	POST /api/products , PATCH /api/products/:id , DELETE /api/products/:id	✅ built
Checkout	POST /api/orders	✅ built
Customer order history/tracking	GET /api/orders/mine	✅ built
Vendor order list	GET /api/orders/vendor	✅ built
Vendor shipment update	PATCH /api/orders/:id/shipment	✅ built
Escrow auto-release after 48hrs	scheduled job, not a request handler	🕒 planned
Vendor review list + submission	GET/POST /api/vendors/:vendorId/reviews	✅ built
Admin report queue + resolve/dismiss	GET /api/reports , PATCH /api/reports/:id/status	✅ built
Vendor's own reports + evidence	GET /api/reports/mine , POST /api/reports/:id/evidence	✅ built
Buyer files a report against an order	POST /api/reports (orderId , reason)	🕒 planned
Vendor payout account	PUT /api/vendor/payout-account , GET /api/vendor/payout-account	🕒 planned
AI shopping assistant	POST /api/assistant/chat	✅ built
Chat (buyer↔vendor messaging)	not built — needs polling, no WebSockets	🕒 planned

The frontend still calls none of this. Every HTML/JS file in `customer/` , `vendor/` , and `admin/` still runs on hard-coded mock data / `localStorage` (with the one exception of `customer/assets/report-issue.js` , which reaches into `admin/assets/data.js` directly rather than calling any of the above — see `DOCUMENTATION.md`). Pointing the frontend at this API is a distinct next phase, best done feature-by-feature rather than all at once.

6. Security — closing the gaps this prototype leaves open

Everything here is flagged in `DOCUMENTATION.md` §9 too; this is the actionable version.

1. **Server-side role enforcement.** Every `/api/admin/*` route must check the session's role before doing anything, full stop. Today's client-side checks (hiding buttons, filtering activity in JS) are UX, not security — assume any of them can be bypassed by calling the endpoint directly.
2. **Password resets shouldn't hand the new password to an admin.** The current UI shows the generated temp password in a panel for the admin to relay manually. A real system emails a reset *link* (or one-time code) straight to the account holder; an admin triggering a reset should never see the resulting credential.
3. **Real email verification for admin invites**, replacing the "type anything" placeholder — generate a code, hash it, email it, compare hashes, expire it after ~15 minutes.
4. **Rate limiting** on auth endpoints (signin, password reset, invite verification) to block brute-forcing.
5. **Input validation & sanitization** server-side for everything — the mock data trusts whatever's typed into a form; a real backend can't.
6. **Least-privilege for Support vs Moderator vs Super Admin**. Decide the actual permission matrix (the front-end's role-gating today only covers "who can see other admins' activity" and "can't delete the last Super Admin" — suspend/approve/reset-password aren't gated by role at all yet) and enforce it in middleware, not per-route ad hoc checks.
7. **Audit log integrity.** Write `activity_log` rows from the server when a mutation happens, never from a client-supplied "log this" call — otherwise a client can fake or omit entries.

7. Suggested build order

Steps 1–5 are done — see `backend/`. What's left is provisioning (step 0, can't be done from outside a cPanel account), wiring the *frontend* to call any of this instead of its mock data, and steps 6–9 below.

0. 🕒 **cPanel Node.js app + MySQL database, provisioned.** Create the Node app via cPanel's "Setup Node.js App," point it at a subdomain or path, create the MySQL database and user through cPanel's MySQL Database Wizard, and confirm `/api/health` is reachable over HTTPS. `backend/README.md`'s "Deploying to Namecheap shared hosting" section is the concrete walkthrough for this step — it's infrastructure inside your hosting account, so it has to happen there, not in this repo.
1. ✅ **Auth foundation** — `users` table, real password hashing, JWT issuing, the three signin flows (buyer/vendor/admin). `backend/src/routes/auth.routes.js`.
2. ✅ **Admin console backend** — the best-specified surface (§5's table). `backend/src/routes/admin.routes.js`.
3. ✅ **Product + order + cart** — `backend/src/routes/products.routes.js`, `orders.routes.js`. 🕒 Still missing from this step: `backend/src/routes/vendors.routes.js` (public `GET /api/vendors` / `GET /api/vendors/:id` — see §5) for `store.html` / `vendors.html`, which currently read the hard-coded `assets/vendors.js` mock instead; and `backend/src/routes/site-banners.routes.js` (§5) for the `dashboard.html` / `explore.html` banner carousel, currently reading `admin/assets/data.js`'s `localStorage` state directly instead of an API.
4. ✅ **Reports + activity log wired end-to-end** — `backend/src/routes/reports.routes.js`, `src/utils/activityLog.js`.

5. ✅ **AI shopping assistant + product search** — `backend/src/routes/assistant.routes.js`, using keyword-search grounding rather than embeddings for this first pass (see §5's note on why).
6. ⌚ **Email/SMS integrations** — verification codes, password reset links, order receipts. Every place this is missing is marked `TODO: email` in the code (`grep -rn "TODO: email" backend/src`).
7. ⌚ **Payments** — Paystack/Flutterwave integration for real checkout (replacing the simulated "Order placed!" flow in both `customer/cart.html` and the homepage's "Buy now" modal).
8. ⌚ **File uploads** — product images, avatars, review photos — to cPanel's file storage initially, with a clear seam to swap in an external object store later (see §2).
9. ⌚ **Chat** — last, and the one piece this hosting target defers rather than just delays: needs polling (not WebSockets) as noted in §2, or a move off shared hosting first if real-time chat becomes a priority sooner than expected.

Not in the original nine steps, worth calling out separately: **rewiring the frontend itself** to call this API instead of its mock data (`admin/assets/data.js`'s `localStorage` calls → `fetch()`), real form submissions on `signin.html` / `signup.html`, an actual chat UI wired to `POST /api/assistant/chat`). None of the backend work above touches a single frontend file — that's a distinct pass, best done feature-by-feature rather than all at once, since each swap is independently testable.

8. What NOT to over-build

- Don't build a permissions system more granular than the three roles the UI already has (Super Admin / Moderator / Support) unless there's a concrete need — the front-end doesn't have UI for anything finer-grained.
- Don't build real-time infrastructure for the admin console — nothing there needs to push updates to an open tab; a page refresh (as today) is fine.
- Don't build multi-currency or multi-region support — the whole site is NGN/Nigeria-specific (phone formats, delivery copy, Paystack/Flutterwave as the natural payment choice).
- Don't stand up a dedicated vector database (Pinecone, etc.) for the AI product search — application-level cosine similarity over MySQL-stored embeddings (§2, §3) handles this fine at marketplace-catalog scale, and a managed vector DB is real infrastructure to pay for and operate that this project doesn't need yet.
- Don't try to force WebSockets onto shared hosting — polling on an interval is a perfectly normal way to fake "real-time" chat for an MVP, and fighting Passenger to hold persistent connections open is time better spent elsewhere.
- **Don't migrate off shared hosting preemptively.** Everything in §9 below is for when a specific constraint is actually hit (connection limits, real traffic, a real need for WebSockets/background jobs) — not "just in case." Namecheap shared hosting is genuinely fine well past MVP for a marketplace this size.

9. Scaling: moving the database off shared hosting, onto a VPS

Signals it's time to consider this, rather than a fixed traffic number: MySQL `max_connections` errors under normal load, the shared CPU/RAM ceiling from §2 actually being hit (slow response times with nothing obviously wrong in the code), a real need for the WebSocket-based chat or background-job scheduling §2 explicitly deferred, or wanting MySQL tuning/extensions (Redis, full-text search config, etc.) a shared cPanel MySQL instance won't give you control over.

Two ways to do this, in order of how much they actually solve: - **(a) Database only** — keep the Node app on cPanel/Passenger, point it at a MySQL instance running on a new VPS instead of the local cPanel one. Cross-host DB latency on every query is a real cost, and it doesn't remove any of \$2's other shared-hosting limits (still no WebSockets, still Passenger-constrained). Treat this as an interim step, not the destination. - **(b) Database and app together** — move both onto the VPS. This is the actual scaling move: it removes the cross-host latency from (a) *and* lifts the WebSocket/background-job constraints from \$2, since you now have root on the box the app runs on. Recommended once you're doing this at all, unless there's a specific reason to split them (e.g. a managed database service instead of self-hosting MySQL on the same VPS).

The rest of this section assumes (b), since it's a superset of (a) — skip step 7 if you genuinely only want to move the database.

0. **Provision the VPS.** Any mainstream provider (DigitalOcean, Linode, Vultr, Hetzner — a \$6-12/mo droplet is plenty to start); or a managed MySQL service (DigitalOcean Managed Databases, AWS RDS) instead of self-hosting MySQL if you'd rather pay a bit more to not own database ops. Ubuntu LTS is a safe default OS choice. Create a non-root sudo user, set up SSH key auth (disable password auth), and configure a firewall (`ufw allow 22,80,443`, nothing else public — especially not MySQL's `3306` unless you have a specific reason to reach it from outside the box).
1. **Install and configure MySQL/MariaDB on the VPS**, matching the major version already in use on cPanel where possible (avoids dump/restore surprises). Create the production database and a dedicated user (not `root`) with a strong, generated password. If the app runs on the same VPS (option b), bind MySQL to `127.0.0.1` only — the app talks to it over localhost, and it's never exposed to the public internet at all, which is strictly better than cPanel's shared-instance model.
2. **Export the data from shared hosting.** Whichever of these the Namecheap plan allows: - SSH access (some shared plans include it): `mysqldump -u <cpanel_db_user> -p --single-transaction --routines --triggers <dbname> > vetra_dump.sql`. `--single-transaction` matters here — it takes a consistent snapshot without locking tables, important if the site is still live and taking orders during the export. - No SSH: cPanel → phpMyAdmin → select the database → **Export** tab → SQL format, "Custom" options with `Add DROP TABLE` and complete-inserts ticked → download. - Either way, this is also a good moment to take a full cPanel Backup Wizard snapshot as a separate safety net, independent of the migration itself.
3. **Transfer the dump to the VPS:** `scp vetra_dump.sql <user>@<vps-ip>:~/` (or upload via the provider's browser file manager if `scp` isn't set up locally yet).
4. **Import on the VPS:** `mysql -u <new_db_user> -p <new_dbname> < vetra_dump.sql`. Then verify — don't just trust a clean exit code. Run matching `SELECT COUNT(*) FROM <table>;` on both the old and new database for every table (`users`, `products`, `orders`, `order_items`, `reports`, `activity_log`, at minimum) and confirm the counts match before treating the new database as authoritative.
5. **Point the backend at the new database.** Nothing in application code changes for this — `backend/src/db.js`'s `mysql2/promise` pool already reads `DB_HOST / DB_PORT / DB_USER / DB_PASSWORD / DB_NAME` from `.env`; update those four values (to `127.0.0.1` if the app is moving to the same VPS, or the VPS's address if only the database moved).
6. **If moving the app too:** deploy `backend/` on the VPS behind Nginx as a reverse proxy, with PM2 (or a systemd unit) keeping the Node process running and restarting it on crash/reboot — this replaces what cPanel's Passenger was doing. Issue a TLS certificate via Let's Encrypt/certbot. The static frontend (plain HTML/CSS/JS, no build step) doesn't have to move at the same time — it can keep being served from the existing Namecheap shared hosting, or move to the VPS too, or go on a CDN — whichever it is, make sure the API's CORS config allows that origin.
7. **Cut over gradually, not by flipping a switch.** Stand up the VPS stack first as a staging target (a test subdomain pointed at it), and run through every real flow against it — signup, signin, checkout, an admin

action — before touching production DNS or the live `.env`. Keep the shared-hosting database untouched and readable for a rollback window after cutover; don't decommission it the same day.

8. **New responsibilities cPanel was quietly handling that a VPS doesn't:** backups (shared hosting typically auto-backs-up; on a VPS, that's now a nightly `mysqldump` cron piped to off-box storage like S3/R2, or your provider's disk-snapshot feature — either way, something you have to set up, not something that already exists), OS and MySQL security patching, database tuning (`innodb_buffer_pool_size` and friends — cPanel's defaults are chosen to be safe across many tenants sharing one box; a dedicated VPS can be tuned for just this app's workload), and monitoring/alerting (uptime, disk usage, slow-query log) since there's no hosting-provider dashboard doing this for you anymore.
9. **Revisit §2's shared-hosting-specific constraints once you're here** — several of them were only true *because* of Passenger/shared MySQL, and stop applying on a VPS with root access: WebSockets become viable, so real chat (§7 step 9) no longer has to be polling-only; the escrow 48-hour auto-release job (§5's `PATCH /api/orders/:id/shipment` note) can become a real scheduled worker instead of waiting on cPanel's constrained cron; and Redis becomes an option for session/rate-limit caching if traffic ever justifies it. None of this needs to happen at migration time — just don't assume the shared-hosting limitations from §2 still apply once they don't.